

OpenAI API Python 실습



Contents

1 주의사항

1.1 API Key 보안

2 OpenAI API Key 발급받기

2.1 OpenAI Platform에 로그인한다.

2.2 Projects 메뉴로 이동

2.3 Billing 메뉴로 이동

2.4 Add payment method 진행

2.5 자동 충전 방식은 필요하면 나중에 설정

2.6 Dashboard 또는 API Keys 메뉴로 이동한다.

2.7 Create new secret key버튼을 선택한다.

2.8 구분하기 쉬운 이름을 입력

2.9 Project명은 아까 만든 프로젝트로 선택

2.10 중요! 생성된 API Key는 다시 확인하기 어려우므로 안전한 위치에 보관

3 API Key 사용

3.1 .env 파일 사용

3.2 .env파일에는 다음 형식으로 작성한다.

3.3 openai와 python-dotenv 설치

3.4 API Key 불러오기 확인

4 OpenAI 클라이언트 만들기

4.1 client 객체 생성

4.2 기본 API 호출

4.3 질문을 구체적으로 작성하기

4.4 대상에 맞는 설명 생성

4.5 자연어를 구조화된 명령으로 변환

5 VLM 실습

5.1 실습용 샘플 이미지 만들기

5.2 로컬 이미지를 Base64로 변환하기

5.3 이미지와 질문을 함께 입력

5.4 로봇 이미지 분석

5.5 VLM 결과를 JSON 형태로 받기

5.6 내가 준비한 이미지로 바꿔보기

5.7 인터넷 이미지 URL 사용

OpenAI API Python 실습

Exported from Confluence · <https://pinkwink.atlassian.net>

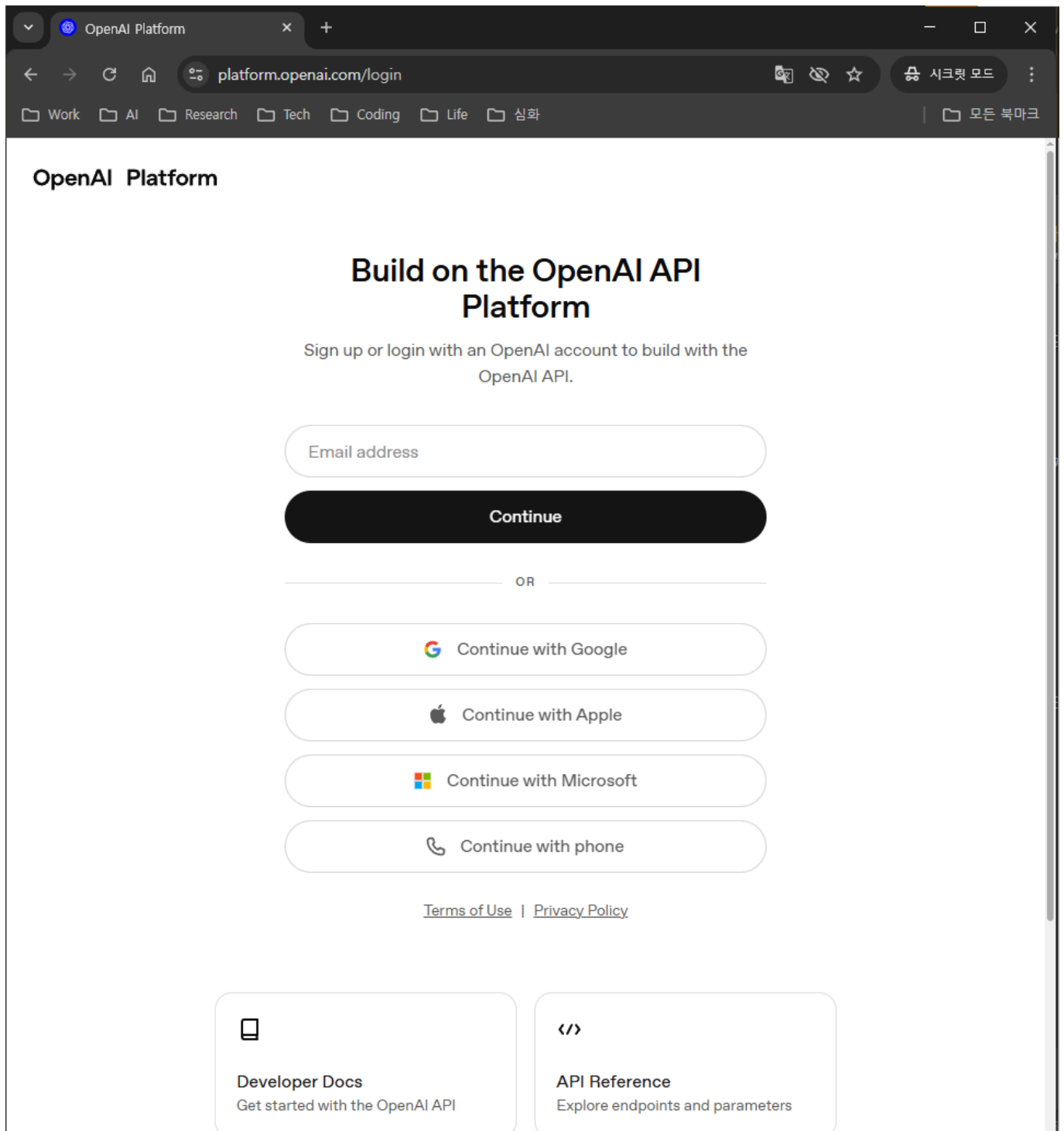
주의사항

API Key 보안

- API Key는 개인 인증 정보에 해당한다.
- API Key를 코드 셀에 직접 작성하지 않는다.
- API Key가 보이는 화면을 캡처하거나 공유하지 않는다.
- API Key가 포함된 파일을 GitHub, Notion, LMS, 메신저 등에 업로드하지 않는다.

OpenAI API Key 발급받기

OpenAI Platform에 로그인한다.



Projects 메뉴로 이동

Projects - OpenAI API

platform.openai.com/settings/organization/projects

API DocsStart building

Settings

Your profile

Organization

General

API keys

Admin keys

People

Projects

Billing

Limits

Usage

Service health

Data controls

Security

Project

General

API keys

Webhooks

Evaluations

People

Projects

Search projects by name...

Geography

Data retention

Active

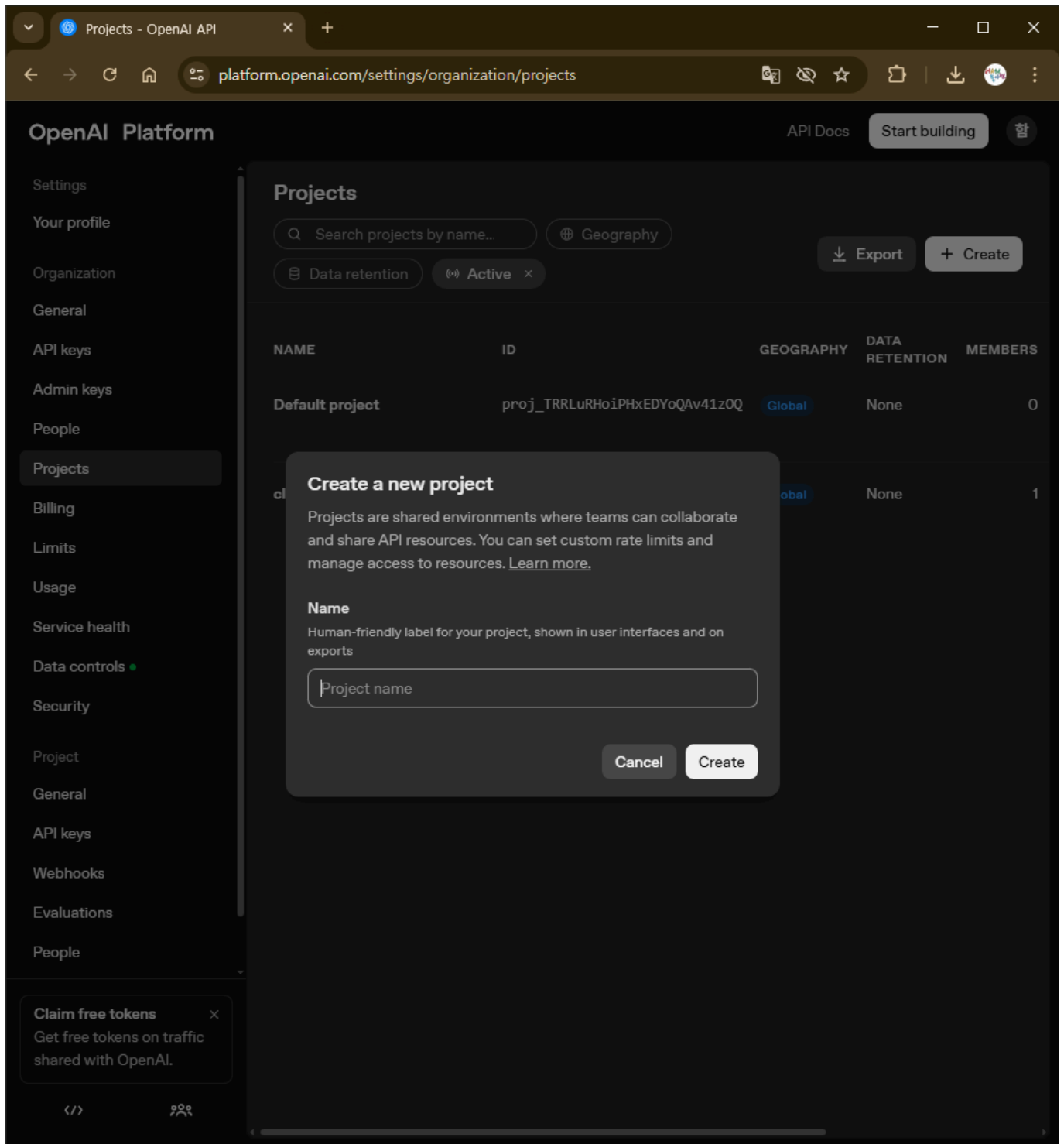
Export

Create

NAME	ID	GEOGRAPHY	DATA RETENTION	MEMBERS
Default project	proj_TRRLuRHoiPHxEDYoQAv41zOQ	Global	None	0
class_260512_AIRobot_proj	proj_YBpOxnWLAQpGghwezPQs34iE	Global	None	1

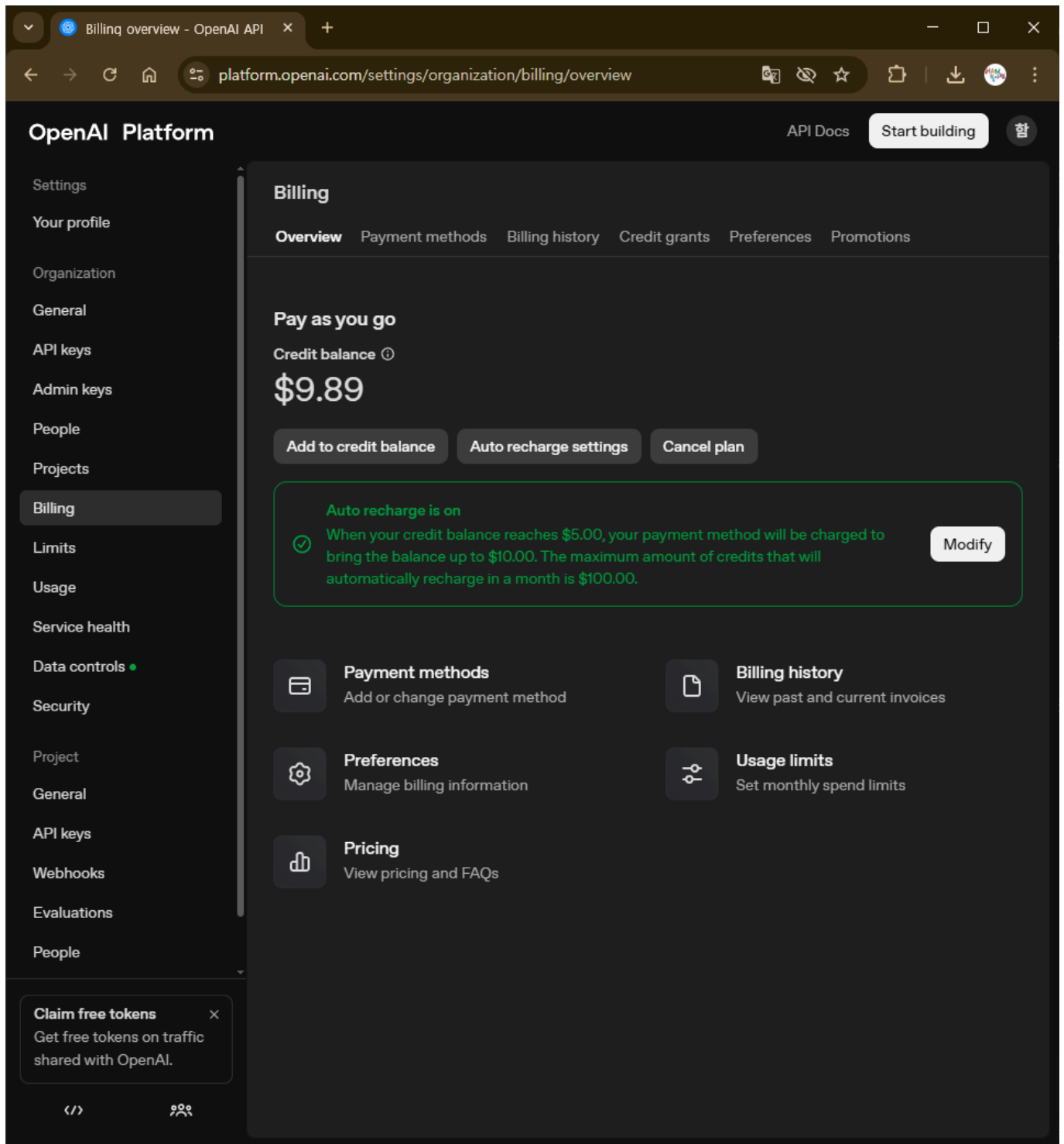
Claim free tokensGet free tokens on traffic shared with OpenAI.

Create 버튼 누르고 새 프로젝트를 만들어주자

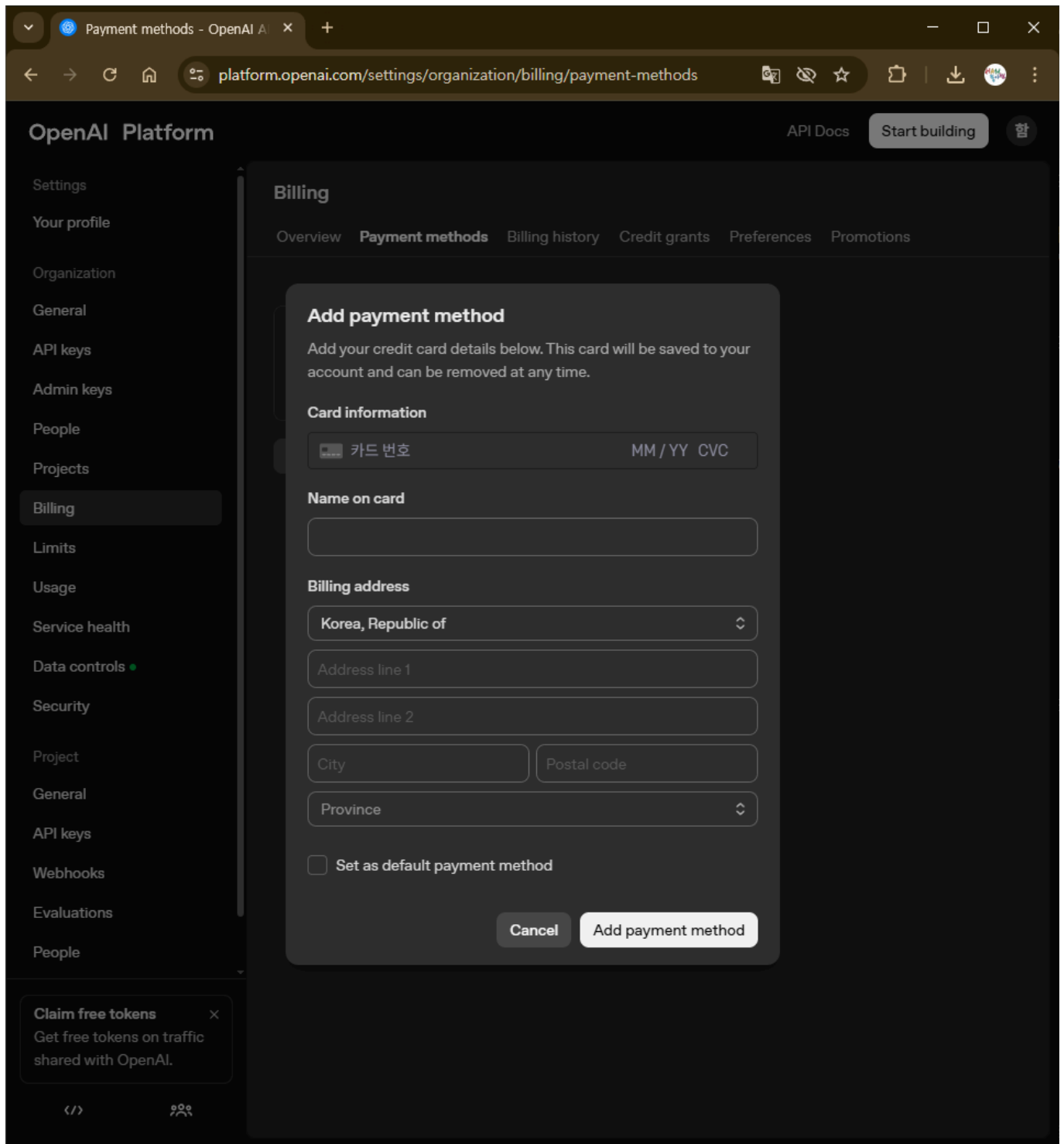


Billing 메뉴로 이동

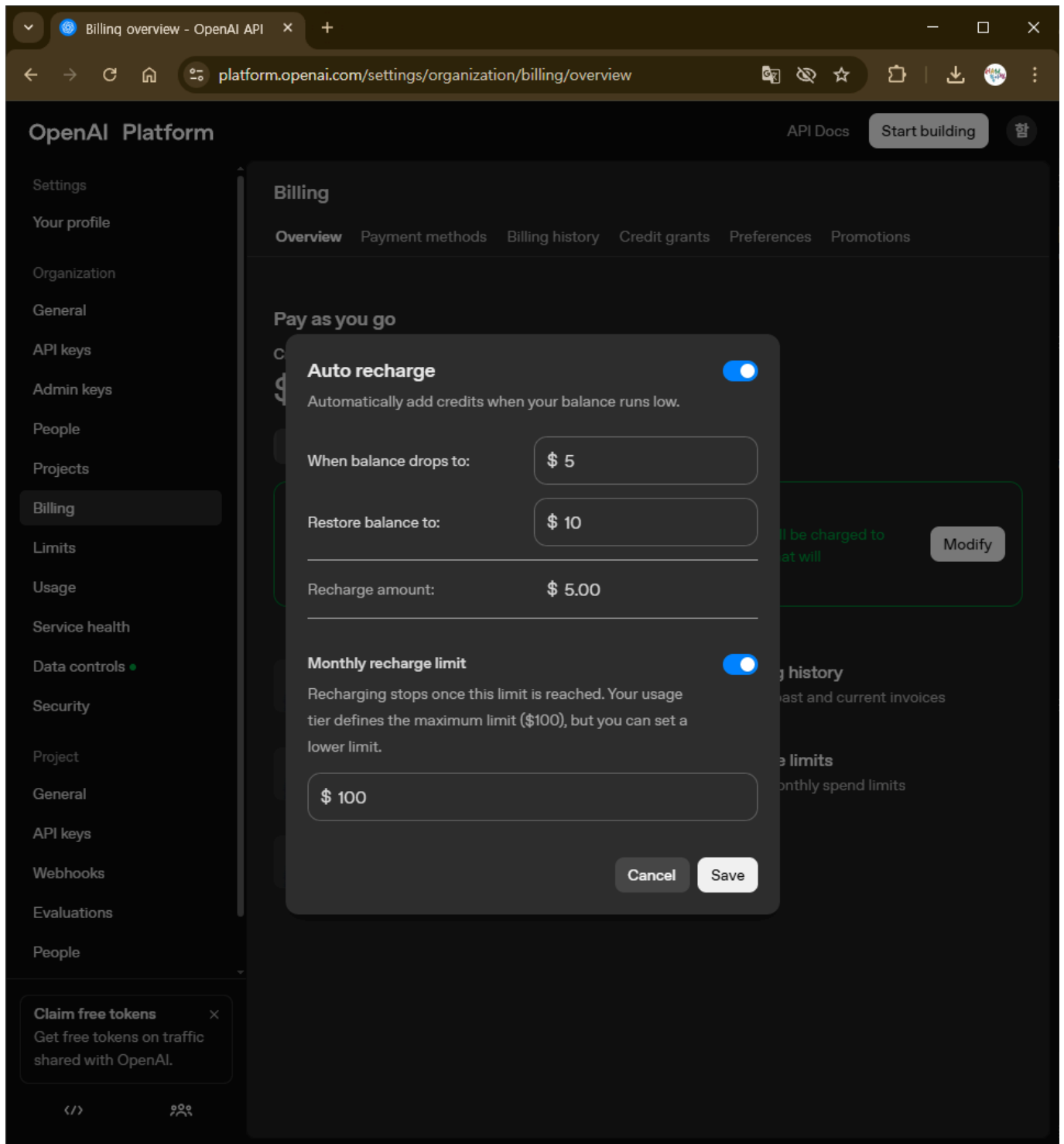
결제를 안 하더라도 payment method 부터 등록해야 API key를 준다.



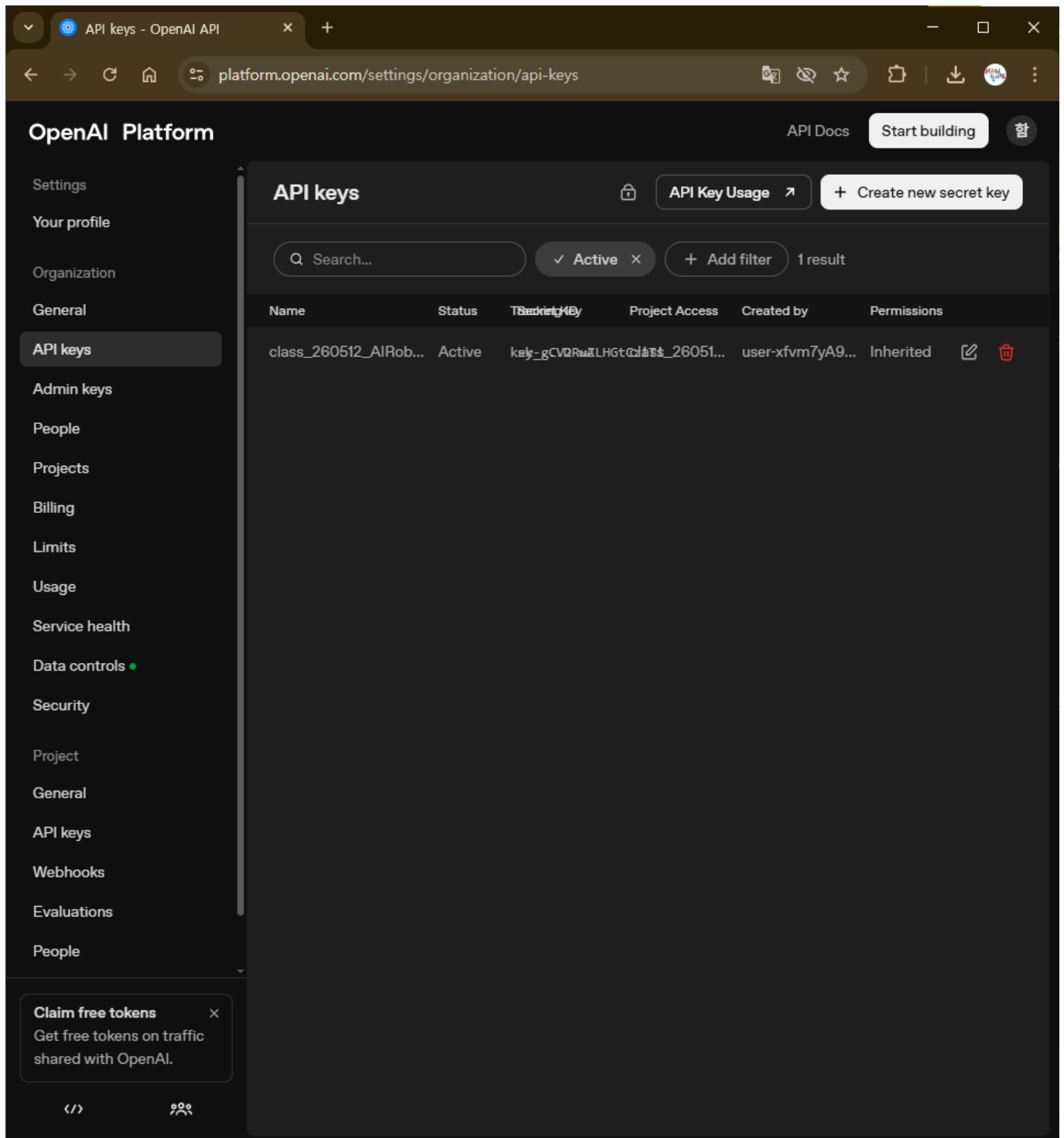
Add payment method 진행



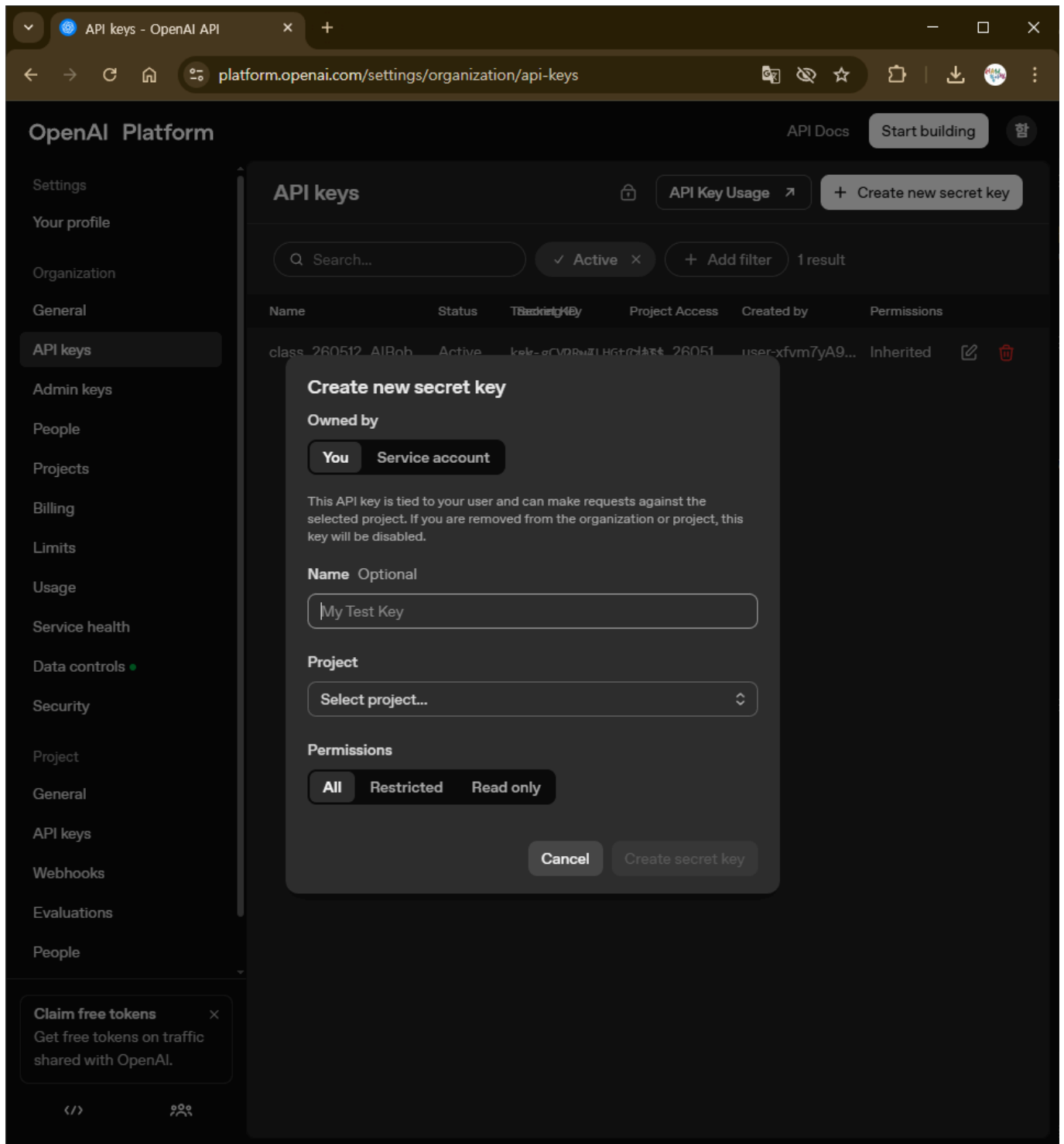
자동 충전 방식은 필요하면 나중에 설정



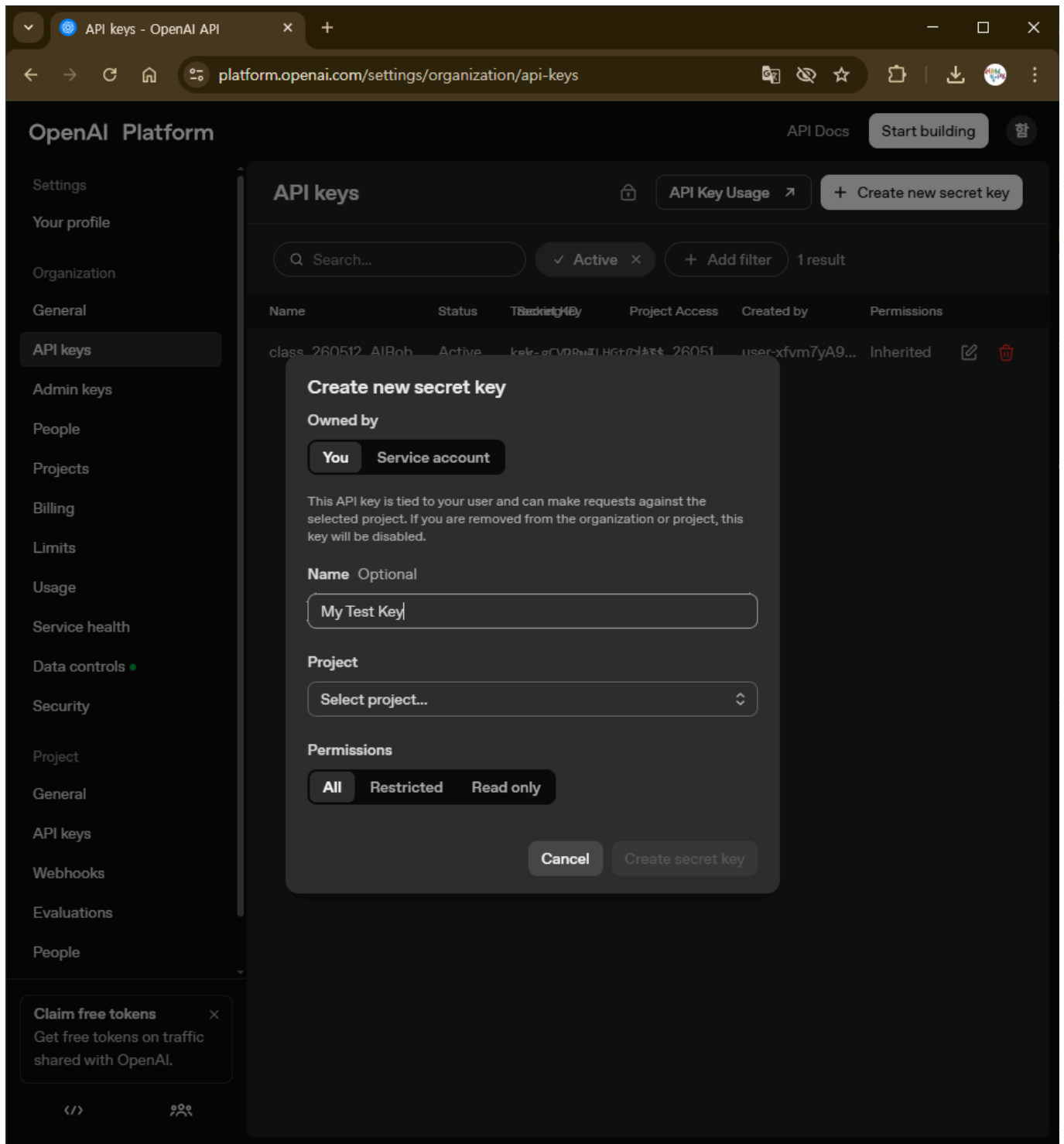
Dashboard 또는 API Keys 메뉴로 이동한다.



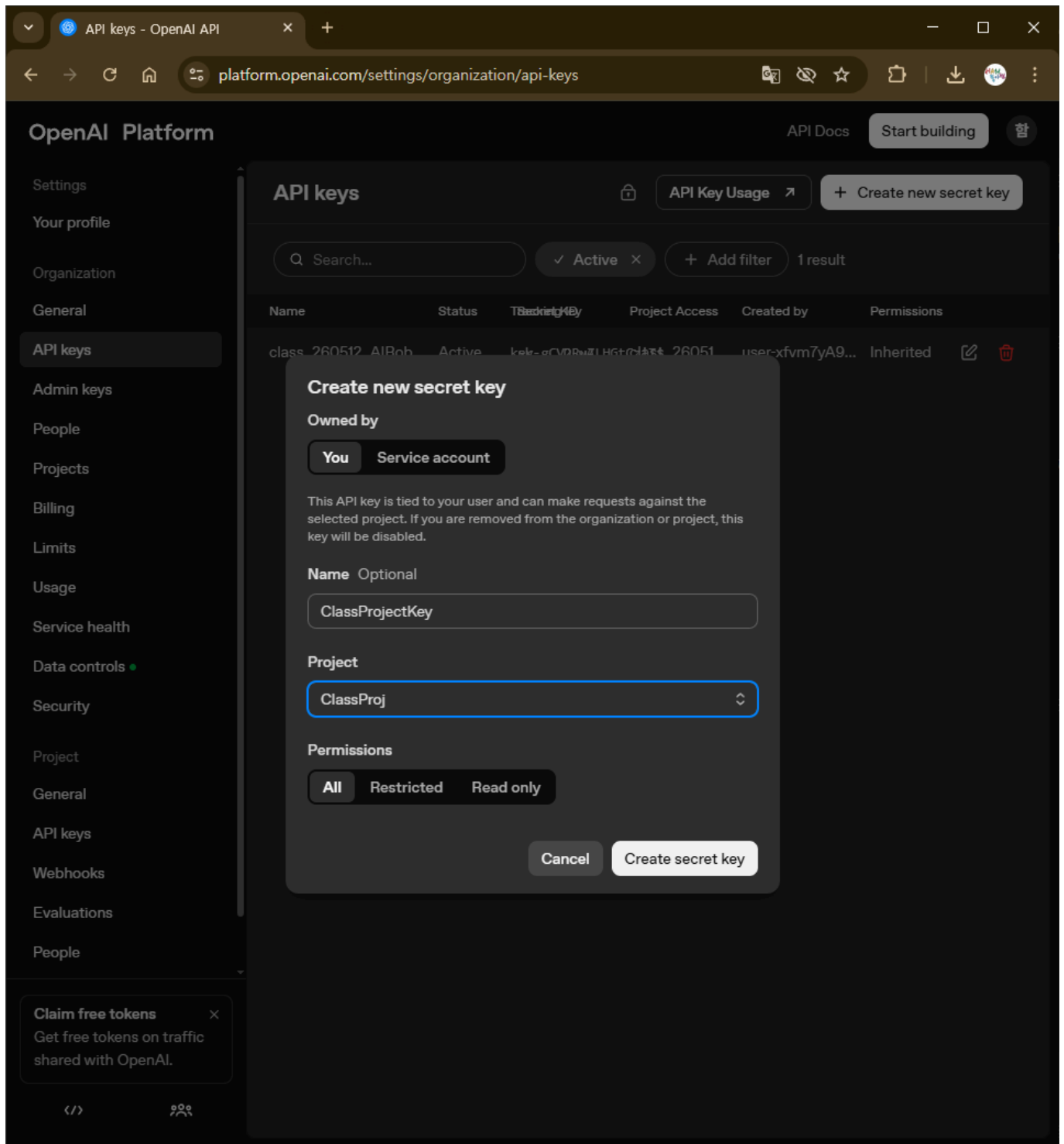
Create new secret key 버튼을 선택한다.



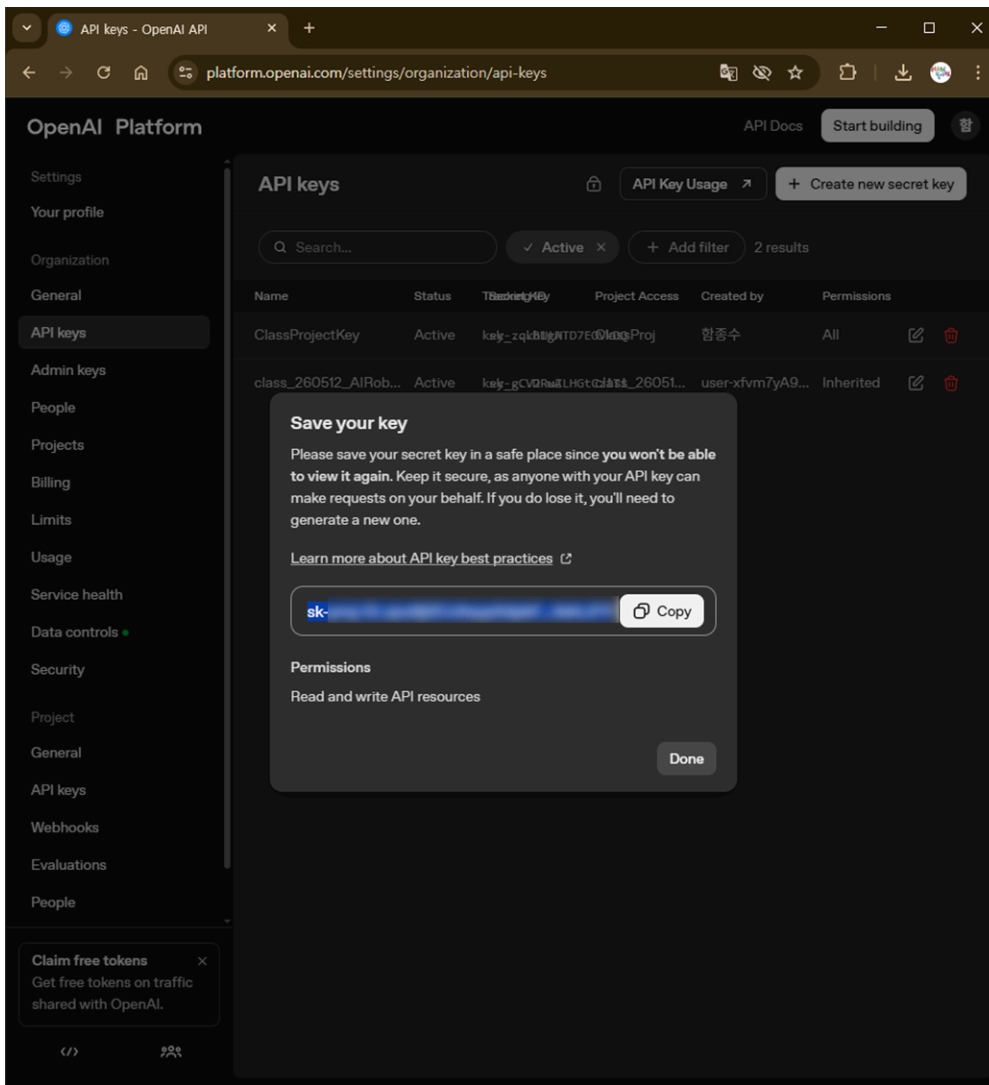
구분하기 쉬운 이름을 입력



Project명은 아까 만든 프로젝트로 선택



중요! 생성된 API Key는 다시 확인하기 어려우므로 안전한 위치에 보관



API Key 사용

.env 파일 사용

- 실행할 코드와 같은 위치에 `.env` 파일을 만든다.

Code

```
1 touch .env
```

`.env` 파일에는 다음 형식으로 작성한다.

등호 앞뒤에는 불필요한 공백을 넣지 않는다.

text

```
1 OPENAI_API_KEY=여기에_본인의_API_KEY
```

openai와 python-dotenv 설치

- `openai` 패키지는 OpenAI API를 Python에서 사용하기 위한 공식 SDK
- `python-dotenv` 패키지는 `.env` 파일의 값을 Python 코드로 불러오기 위한 도구

Code

```
1 pip install -q openai python-dotenv
```

API Key 불러오기 확인

```
[2]: import os
from pathlib import Path
from dotenv import load_dotenv

env_path = Path(".env")

if not env_path.exists():
    print(".env 파일이 없습니다.")
    print("노트북 파일과 같은 폴더에 .env 파일을 만들고 아래처럼 작성하세요.")
    print("OPENAI_API_KEY=여기에_본인의_API_KEY")
else:
    load_dotenv(dotenv_path=env_path, override=True)

api_key = os.environ.get("OPENAI_API_KEY")

if api_key:
    masked = api_key[:7] + "... + api_key[-4:]
    print("OPENAI_API_KEY를 .env에서 불러왔습니다.")
    print("확인용 일부 표시:", masked)
else:
    print(".env 파일은 있지만 OPENAI_API_KEY 값이 없습니다.")
    print(".env 파일 안에 아래 형식으로 작성했는지 확인하세요.")
    print("OPENAI_API_KEY=여기에_본인의_API_KEY")
```

OPENAI_API_KEY를 .env에서 불러왔습니다.
확인용 일부 표시: sk-svca...D-wA

OpenAI 클라이언트 만들기

client 객체 생성


```
[3]: from openai import OpenAI
from dotenv import load_dotenv

load_dotenv(dotenv_path=".env", override=True)

client = OpenAI()

MODEL_NAME = "gpt-5.5"

print("OpenAI 클라이언트 준비 완료")
print("사용 모델:", MODEL_NAME)
```

OpenAI 클라이언트 준비 완료
사용 모델: gpt-5.5

기본 API 호출

```
[4]: response = client.responses.create(
    model=MODEL_NAME,
    input="로봇을 처음 배우는 학생에게 ROS2를 한 문장으로 설명해줘."
)

print(response.output_text)
```

ROS2는 로봇의 센서, 모터, 카메라 같은 여러 부품들이 서로 데이터를 주고받으며 함께 동작하도록 도와주는 로봇용 소프트웨어 프레임워크입니다.

질문을 구체적으로 작성하기

좋은 답변을 얻으려면 목적, 대상, 형식, 길이, 조건을 구체적으로 작성한다.

[5]: `prompt = """
고등학생이 이해할 수 있게 API를 설명해줘.`

`조건:`

- `1. 배달앱 주문 과정에 비유하기`
- `2. 어려운 용어는 피하기`
- `3. 마지막에 한 줄 요약 넣기`

`"""`

```
response = client.responses.create(  
    model=MODEL_NAME,  
    input=prompt  
)
```

```
print(response.output_text)
```

API는 ****서로 다른 프로그램끼리 정해진 방식으로 대화하게 해주는 연결 통로****라고 생각하면 돼요.

배달앱 주문 과정으로 비유해볼게요.

- **사용자 = 손님****
손님이 배달앱에서 치킨을 고릅니다.
- **배달앱 = 요청을 보내는 쪽****
배달앱은 “이 손님이 치킨 1마리를 주문했어요”라는 정보를 음식점에 전달합니다.
- **API = 배달앱과 음식점 사이의 약속된 주문 방식****
음식점마다 아무 말로나 주문하면 헛갈리겠죠?
그래서 “메뉴 이름, 수량, 주소, 결제 정보”처럼 정해진 형식으로 주문을 보내야 합니다.
이처럼 프로그램끼리 정보를 주고받을 때 사용하는 약속이 API입니다.
- **음식점 = 요청을 처리하는 쪽****
음식점은 주문 내용을 확인하고 “주문 접수 완료” 또는 “재료가 없어서 주문 불가” 같은 답을 보냅니다.
- **결과 = 손님에게 보이는 화면****
배달앱은 음식점의 답을 받아서 사용자에게 “주문이 완료되었습니다”라고 보여줍니다.

예를 들어 날씨 앱을 켜를 때 현재 날씨가 보이는 것도 비슷해요.

날씨 앱이 날씨 정보를 가진 곳에 “서울의 날씨 알려줘”라고 요청하고, 그곳이 “현재 22도, 맑음”이라고 답해주는 식입니다.

****한 줄 요약: API는 배달앱이 음식점에 정해진 방식으로 주문을 넣듯, 프로그램끼리 정보를 주고받게 해주는 약속된 통로입니다.****

대상에 맞는 설명 생성

같은 주제라도 대상에 따라 설명 방식이 달라져야 한다.

```
[6]: topic = "API"

prompt = f"""
{topic}를 세 가지 수준으로 설명해줘.
```

1. 초등학생용
2. 고등학생용
3. 개발자용

각 설명은 3문장 이내로 작성해줘.
"""

```
response = client.responses.create(
    model=MODEL_NAME,
    input=prompt
)

print(response.output_text)
```

1. 초등학생용

API는 서로 다른 프로그램들이 “말을 주고받는 약속”이에요.
예를 들어, 날씨 앱이 날씨 회사에 “오늘 날씨 알려줘”라고 물어보는 방법이에요.

2. 고등학생용

API는 프로그램끼리 데이터를 요청하고 응답받기 위한 정해진 인터페이스입니다.
예를 들어, 앱이 지도 API를 사용하면 직접 지도를 만들지 않아도 위치 검색이나 길찾기 기능을 사용할 수 있습니다.

3. 개발자용

API는 소프트웨어 컴포넌트 간 상호작용을 정의한 명세로, 요청 형식, 응답 형식, 인증 방식, 엔드포인트 등을 포함합니다.
예를 들어 REST API에서는 클라이언트가 HTTP 요청을 보내고 서버가 JSON 같은 형식으로 데이터를 반환합니다.

자연어를 구조화된 명령으로 변환

```
[7]: user_command = "로봇아, 앞으로 천천히 가다가 장애물이 보이면 멈춰."
```

```
prompt = f"""
다음 사용자의 말을 로봇 제어 명령으로 해석해줘.
```

사용자 말:
{user_command}

출력 형식:
동작:
속도:
정지 조건:
주의사항:
"""

```
response = client.responses.create(
    model=MODEL_NAME,
    input=prompt
)

print(response.output_text)
```

동작: 앞으로 이동

속도: 천천히

정지 조건: 전방에 장애물이 감지되면 즉시 정지

주의사항: 장애물 감지 센서를 확인하며 안전거리를 유지할 것

VLM 실습

실습용 샘플 이미지 만들기

```
[8]: from pathlib import Path
import base64
from IPython.display import Image, display

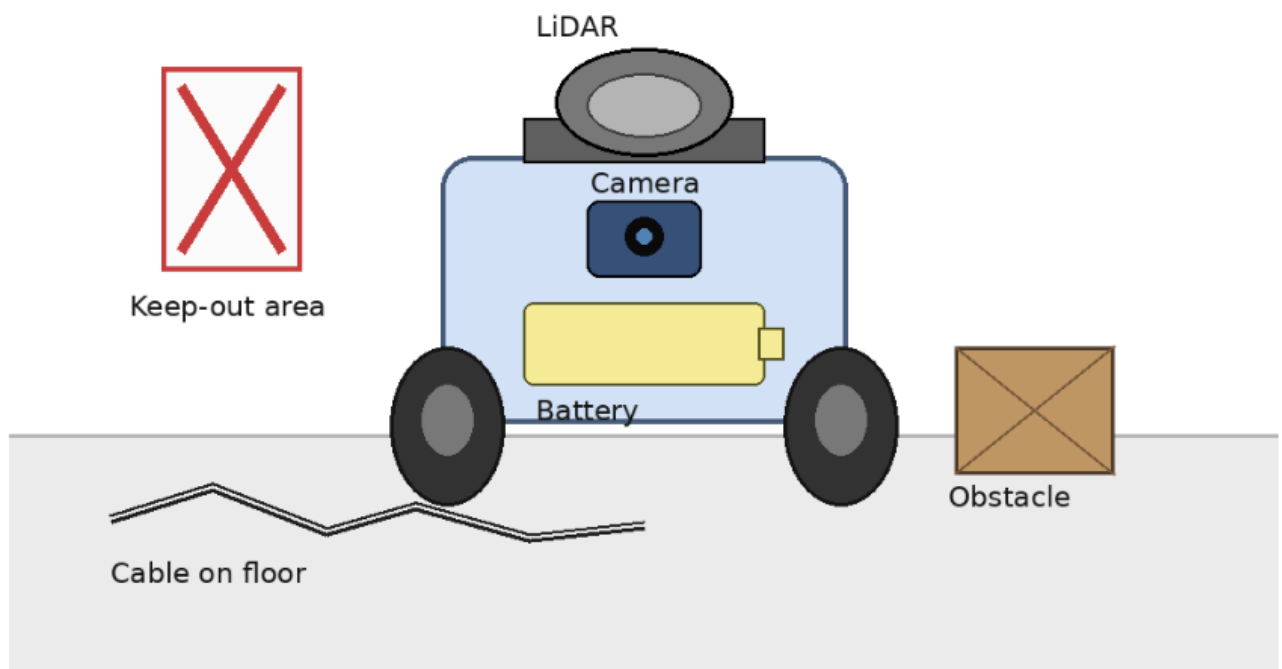
sample_image_base64 = """iVBORw0KGgoAAAANSUhEUgAAA+gAAAJsCAIAAADh1x7JAABkw01EQVR4nO3dd3wU1f7/8bMBUgg1ECABQp0miBpQpCtC

sample_image_path = Path("sample_robot_lab.png")
sample_image_path.write_bytes(base64.b64decode(sample_image_base64))

print("샘플 이미지 생성 완료:", sample_image_path)
display(Image(filename=str(sample_image_path)))
```

샘플 이미지 생성 완료: sample_robot_lab.png

Robot Lab Safety Image



로컬 이미지를 Base64로 변환하기

컴퓨터에 저장된 이미지는 API 입력에 바로 넣을 수 없다.

```
[9]: import base64
from pathlib import Path

image_path = Path("sample_robot_lab.png")

image_bytes = image_path.read_bytes()
base64_image = base64.b64encode(image_bytes).decode("utf-8")

print("이미지 파일:", image_path)
print("Base64 길이:", len(base64_image))
print("앞부분 미리보기:", base64_image[:50] + "...")
```

이미지 파일: sample_robot_lab.png

Base64 길이: 34472

앞부분 미리보기: iVBORw0KGgoAAAANSUhEUGAAA+gAAAJsCAIAAADh1x7JAABkw0...

이미지와 질문을 함께 입력

VLM은 Vision-Language Model의 약어

```
[11]: response = client.responses.create(
    model=MODEL_NAME,
    input=[
        {
            "role": "user",
            "content": [
                {
                    "type": "input_text",
                    "text": "이 이미지를 한국어로 쉽게 설명해줘. 로봇 실습 관점에서 중요한 부분을 중심으로 설명해줘."
                },
                {
                    "type": "input_image",
                    "image_url": f"data:image/png;base64,{base64_image}"
                }
            ]
        }
    ]
)

print(response.output_text)
```

이 그림은 **로봇 실습실에서 로봇을 운용할 때 주의해야 할 안전 요소**를 보여줍니다.

1. 로봇 주요 부품

LiDAR

- 로봇 위쪽에 있는 원형 센서입니다.
- 주변 물체와 사람을 감지해서 로봇이 장애물을 피하거나 위치를 파악하는 데 사용됩니다.
- 실습 중에는 **LiDAR를 손으로 가리거나 물건으로 막으면 안 됩니다.**

Camera

- 로봇 앞쪽에 있는 카메라입니다.
- 물체 인식, 사람 인식, 주행 경로 확인 등에 사용됩니다.
- 렌즈가 더럽거나 가려지면 로봇이 주변을 제대로 인식하지 못할 수 있습니다.

Battery

- 로봇의 전원입니다.
- 배터리 연결 상태를 확인해야 하며, 충전 중이거나 사용 중일 때無理하게 만지면 위험할 수 있습니다.
- 배터리가 부풀거나 뜨거워지면 즉시 사용을 중지해야 합니다.

Wheels

- 로봇이 움직이는 바퀴입니다.
- 로봇이 작동 중일 때 바퀴 근처에 손이나 발을 가까이 대면 끼일 수 있습니다.

2. 실습 안전에서 중요한 위험 요소

로봇 이미지 분석

단순 이미지 설명보다 분석 기준을 구체적으로 제시하면 더 유용한 결과를 얻을 수 있다.

```
[12]: prompt = """
이 이미지는 로봇 교육 실습용 이미지다.

다음 기준으로 한국어로 분석해줘.

1. 보이는 주요 물체
2. 로봇 부품으로 보이는 것
3. 각 부품의 예상 역할
4. 로봇 주행에 방해가 될 수 있는 요소
5. 안전상 주의해야 할 점
"""

response = client.responses.create(
    model=MODEL_NAME,
    input=[
        {
            "role": "user",
            "content": [
                {"type": "input_text", "text": prompt},
                {"type": "input_image", "image_url": f"data:image/png;base64,{base64_image}"}
            ]
        }
    ]
)

print(response.output_text)
```

다음 이미지는 로봇 실습실에서 주행 로봇과 주변 위험 요소를 설명하기 위한 교육용 그림으로 보입니다.

1. 보이는 주요 물체

- 이동형 로봇 본체
- 바퀴 2개
- 상단의 LiDAR 센서
- 전면의 카메라
- 내부 또는 측면에 표시된 배터리
- 바닥에 놓인 케이블
- 오른쪽의 장애물 상자
- 왼쪽의 출입 금지 또는 접근 금지 구역 표시
- 실습실 바닥과 벽면 경계선

2. 로봇 부품으로 보이는 것

VLM 결과를 JSON 형태로 받기

```
[13]: import json
```



```
prompt = """
이미지를 분석해서 반드시 JSON 형식으로만 답해줘.
마크다운 코드블록은 쓰지 마.
```

```
형식:
```

```
{
  "summary": "이미지 전체 요약",
  "objects": ["보이는 물체 목록"],
  "robot_parts": ["로봇 부품으로 보이는 것"],
  "safety_risks": ["위험 요소"],
  "recommended_action": "실습 전 추천 행동"
}
```

```
response = client.responses.create(
    model=MODEL_NAME,
    input=[
        {
            "role": "user",
            "content": [
                {"type": "input_text", "text": prompt},
                {"type": "input_image", "image_url": f"data:image/png;base64,{base64_image}"}
            ]
        }
    ]
)
```

```
raw_text = response.output_text
print(raw_text)
```

```
print("\n--- JSON 파싱 시도 ---")
```

```
try:
    data = json.loads(raw_text)
    print("JSON 파싱 성공")
    print("위험 요소:", data.get("safety_risks"))
except json.JSONDecodeError:
    print("JSON 파싱 실패")
    print("모델이 JSON 이외의 문장을 썼을 수 있습니다. 프롬프트를 더 강하게 수정해보세요.")
```

```
{
  "summary": "로봇 실습실 안전 안내 이미지로, 바퀴 달린 이동 로봇 주변에 출입 금지 구역, 바닥 케이블, 장애물이 표시되어 있다.",
  "objects": ["이동 로봇", "Keep-out area 표시", "바닥 케이블", "장애물 상자", "바닥", "안전 안내 제목"],
  "robot_parts": ["LiDAR", "카메라", "배터리", "바퀴", "로봇 본체"],
  "safety_risks": ["이동 로봇의 LiDAR 센서 범위에 장애물 상자가 포함되어 충돌 위험이 있습니다.", "바닥에 노출된 케이블은 로봇의 바퀴에 걸릴 위험이 있습니다.", "출입 금지 구역이 표시되어 있지만 로봇이 이를 인식하지 못할 수 있습니다."],
  "recommended_action": "로봇이 실습실 안으로 진입하기 전에 이미지를 다시 확인하고, 장애물 상자를 제거하고, 바닥 케이블을 정리하십시오."
}
```

내가 준비한 이미지로 바꿔보기

```
[14]: from pathlib import Path
import base64
from IPython.display import Image, display

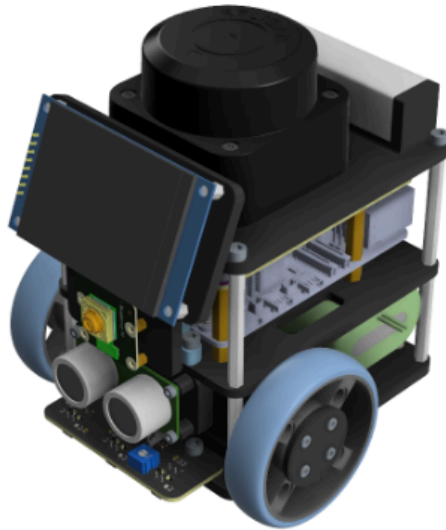
your_image_path = Path("pinky_pro_image.png")

if not your_image_path.exists():
    print("my_robot.jpg 파일이 없습니다.")
    print("직접 분석할 이미지를 노트북과 같은 폴더에 넣고 파일명을 수정하세요.")
else:
    display(Image(filename=str(your_image_path)))

your_base64_image = base64.b64encode(your_image_path.read_bytes()).decode("utf-8")

response = client.responses.create(
    model=MODEL_NAME,
    input=[
        {
            "role": "user",
            "content": [
                {"type": "input_text", "text": "이 사진에서 로봇 부품, 센서, 위험 요소를 구분해서 설명해줘."},
                {"type": "input_image", "image_url": f"data:image/jpeg;base64,{your_base64_image}"}
            ]
        }
    ]
)

print(response.output_text)
```



인터넷 이미지 URL 사용

접근 권한이 필요한 이미지는 모델이 불러오지 못할 수 있다.


```
[16]: image_url = "https://github.com/pinklab-art/pinky_pro/blob/main/doc/pinky_pro_image.png?raw=true"

if image_url == "이미지_URL을_여기에_입력":
    print("먼저 image_url 변수에 분석할 이미지 URL을 입력하세요.")
else:
    response = client.responses.create(
        model=MODEL_NAME,
        input=[
            {
                "role": "user",
                "content": [
                    {"type": "input_text", "text": "이 이미지에 무엇이 보이는지 한국어로 설명해줘."},
                    {"type": "input_image", "image_url": image_url}
                ]
            }
        ]
    )

    print(response.output_text)
```

이미지에는 작은 자율주행 로봇 또는 교육용 로봇 플랫폼의 3D 렌더링이 보입니다.

- 양쪽에 큰 바퀴가 달린 2륜 이동 로봇 형태입니다.
- 앞쪽에는 초음파 센서처럼 보이는 두 개의 원통형 센서가 장착되어 있습니다.
- 전면 상단에는 작은 디스플레이 또는 카메라 모듈처럼 보이는 검은색 사각 패널이 기울어져 있습니다.
- 상단에는 원형의 검은 장치가 있는데, 라이다(LiDAR) 센서나 회전형 센서처럼 보입니다.
- 내부에는 회로기판, 커넥터, 배터리로 보이는 부품들이 층 구조로 배치되어 있습니다.
- 금속 지지대와 검은색 플레이트로 프레임이 구성되어 있으며, 전체적으로 로봇 제작 키트나 연구용 모바일 로봇처럼 보입니다.



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio